

MLOps Face Recognition System

High-Level Design Document

Document Type: Design & Implementation Review — HLD

System: Siamese Face Recognition — Full MLOps Pipeline

Components: Flask API · Airflow DAG · Trainer · MLflow · Prometheus/Grafana · Docker Compose

Technology: PyTorch · Optuna · Airflow 3.1.8 · MLflow 2.11.1 · Docker Compose

April 2026

1 Introduction & System Overview

This document presents the **High-Level Design (HLD)** for a production-grade MLOps pipeline built around a Siamese Neural Network for real-time face recognition. It narrates the architectural decisions made at the system level, explains the design rationale, and provides the conceptual blueprint that guided implementation. The companion documents cover the Low-Level Design (LLD), API specifications, and test plans respectively.

1.1 Purpose & Scope

The primary objective of this system is to deliver a **self-improving, privacy-preserving face recognition service** that can be deployed on commodity hardware — with optional remote-GPU offload — and maintained entirely through automated pipelines. The scope encompasses six tightly orchestrated Docker Compose subsystems covering inference serving, model training, experiment tracking, workflow scheduling, observability, and persistent storage.

1.2 Design Philosophy

Three overarching principles guided every architectural decision in this system:

Separation of Concerns: The UI, inference engine, training pipeline, and observability stack are independent software planes communicating only through well-defined interfaces (REST, MLflow registry, DAG triggers). Each plane can be modified, scaled, or replaced without cascading changes.

Operational Autonomy: Once deployed, the system detects model degradation via human-in-the-loop (HITL) misclassification feedback and triggers its own re-training cycle without manual intervention. A weekly fallback schedule ensures the model stays current even in low-traffic deployments.

Reproducibility & Auditability: All experiment hyperparameters, metrics, and artefacts are tracked in MLflow. A DVC pipeline captures data lineage. Every training run can be fully reproduced from params.yaml and the registered dataset revision.

2 Architecture Overview — The Three-Plane Model

The system is organised into **three primary functional planes**, each with distinct responsibilities, plus a cross-cutting **Observability Plane**. The planes are connected exclusively via REST endpoints, the MLflow Model Registry, and Airflow DAG trigger calls — there is *no shared memory, no direct function calls, and no coupled deployment dependencies* between them.

Plane	Core Services	Primary Role	Communication Interface
Inference Plane	Flask API (flask-api : 2000/8080)	Real-time face detection, embedding extraction, cosine matching, HITL misclassification capture	REST/MJPEG inbound; MLflow read; Airflow trigger outbound

Plane	Core Services	Primary Role	Communication Interface
Training Plane	Airflow DAG + Docker Trainer + Optuna	Scheduled & event-driven model training; hyperparameter optimisation; model registration	DockerOperator; MLflow write; Flask /reload notify
Registry Layer	MLflow Server (port 5000)	Experiment logging; Staging → Production promotion; artefact storage	HTTP from Trainer (write) and Flask API (read)
Observability Plane	Prometheus + Grafana + node/dcgm exporters	System & GPU metrics scraping; recognition latency histograms; SMTP alerting	Scrape /metrics endpoint; dashboards on :3001

Table 1 — The three functional planes and their communication contracts

2.1 Why Three Planes?

Separating inference from training is the single most consequential architectural decision. In a naive monolithic design, re-training would require taking the inference service offline. By decoupling the two via the MLflow Model Registry, the inference plane can continue serving requests while the training plane runs an Optuna hyperparameter sweep in a separate Docker container. When the new model is promoted to *Production* in MLflow, the Flask API receives a webhook notification (/notify_flask) and hot-reloads the model — achieving a **zero-downtime model upgrade**.

2.2 Subsystem Summary

Six Docker Compose services underpin the three planes. Each service has a clearly delineated responsibility and communicates only through the declared interfaces:

Service	Technology Stack	Role
flask-api	Flask 2.x, PyTorch, MediaPipe, SQLite	Live inference, face registration, HITL misclassification capture, Prometheus metrics exposure
trainer	PyTorch, Optuna, MLflow	Siamese network training via sweep_optuna.py or main.py; logs experiments to MLflow; supports local and remote GPU
mlflow-server	MLflow 2.11.1, SQLite backend	Experiment tracking, model registry (Staging / Production), artefact store
airflow (webserver + scheduler + dag-processor)	Apache Airflow 3.1.8, DockerOperator / BashOperator	Weekly pipeline scheduling; misclassification-triggered re-training; model promotion orchestration
monitoring stack	Prometheus, Grafana, node-exporter, dcgm-exporter	System & GPU metrics; recognition latency histograms; alert routing via SMTP

Service	Technology Stack	Role
databases	PostgreSQL 13 (Airflow), SQLite (faces + misclassifications)	Airflow metadata; face embedding store; misclassification audit log

Table 2 — Subsystem summary with technology stack and roles

3

High-Level Design Diagram & Explanation

Figure 1 below is the High-Level Design diagram. It visualises the three planes as nested containers, with data-flow arrows illustrating how information moves between them. Read the diagram from top to bottom: the User/Camera feeds REST and MJPEG streams into the Inference Plane; misclassification events propagate to the Training Plane via the Airflow DAG trigger; the newly trained model flows back through the MLflow registry; and the Observability Plane scrapes metrics from all runtime services.

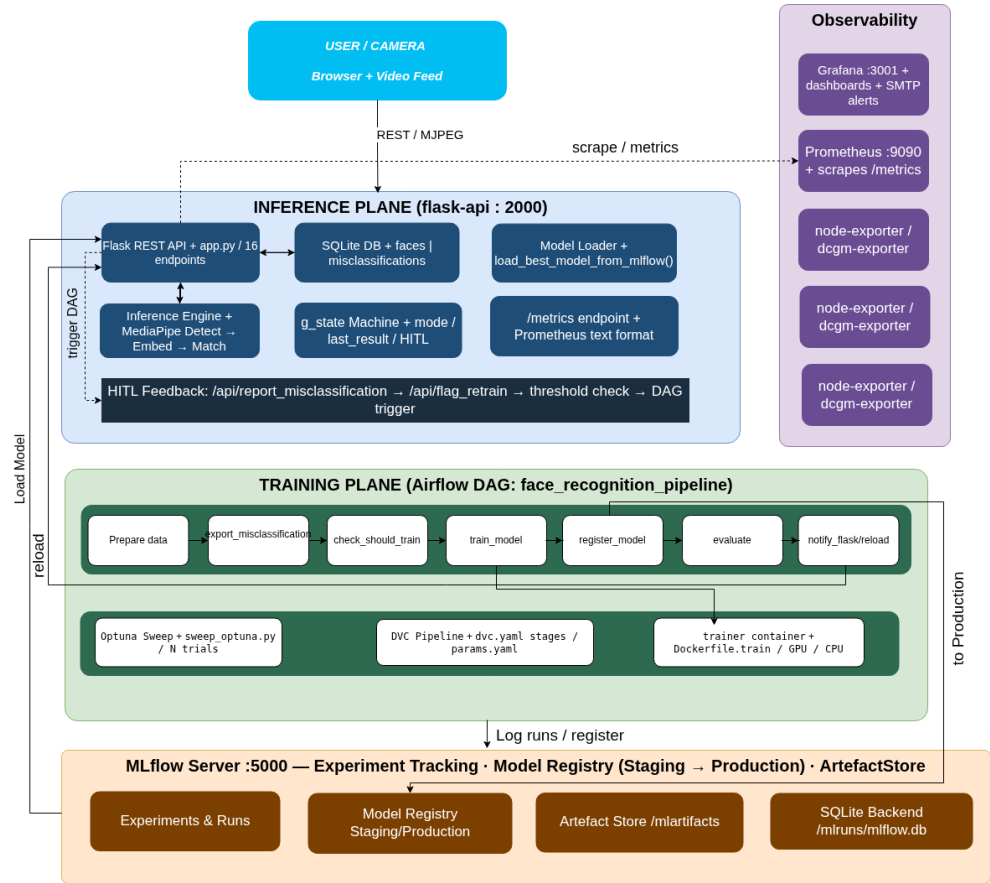


Figure 1 — High-Level Design Diagram: three planes connected via REST, MLflow registry, and DAG triggers

3.1 Inference Plane — What the Diagram Shows

The **Inference Plane** (central blue box, labelled *flask-api : 2000*) contains all components required for real-time recognition. `app.py` exposes 16 REST endpoints. Internally, the video loop (`gen_frames()`) feeds frames from the camera into MediaPipe's face detector, which crops and forwards each detected face to the ResNet50 encoder for embedding extraction. The cosine similarity matcher then queries the SQLite face database and returns the closest identity above the configured threshold. The HITL feedback pathway — visible at the bottom of the inference box — accepts misclassification reports from the user, logs them to SQLite, and fires a JWT-authenticated DAG trigger to Airflow if the configurable threshold is exceeded.

3.2 Training Plane — What the Diagram Shows

The **Training Plane** (lower green box) is orchestrated by the Airflow `face_recognition_pipeline` DAG. The seven sequential tasks visible in the diagram are: `prepare_data` → `export_misclassified` → `check_should_train` → `train_model` → `register_model` → `evaluate_and_promote` → `notify_flask/reload`. Within the training plane, the Optuna sweep block (`sweep_optuna.py`, N trials) runs inside the trainer container (launched by `DockerOperator`), while the DVC pipeline block (`dvc.yaml` stages + `params.yaml`) ensures data and artefact lineage is tracked. After training completes, `register_model.py` pushes the best checkpoint to MLflow, and `evaluate_and_promote()` moves it to Production if the evaluation metric improves on the current baseline.

3.3 MLflow Registry — The Coordination Layer

The **MLflow Server** (bottom orange row) acts as the handshake point between the training and inference planes. It stores experiment runs, model checkpoints, and artefacts under `/mlartifacts`. The registry maintains two model stages: *Staging* (post-training candidate) and *Production* (currently active). After promotion, the Flask API receives a `/notify_flask` call, hot-reloads the new Production model via `load_best_model_from_mlflow()`, and then performs a full re-embedding pass over all registered faces to correct the embedding shift introduced by the updated weights.

3.4 Observability Plane

The **Observability Plane** (right column) operates independently of the other planes. Prometheus scrapes the `/metrics` endpoint of the Flask API every 15 seconds, collecting recognition latency histograms and misclassification Gauges. `node-exporter` and `dcm-exporter` (per host) provide CPU/memory and NVIDIA GPU metrics respectively. Grafana reads Prometheus as its data source and renders preconfigured dashboards on port 3001, with SMTP-based alert routing for threshold violations.

4

Model Foundation & Design Rationale

4.1 ResNet50 Backbone — Rationale

The decision to base the embedding network on a **pre-trained ResNet50** rather than a custom CNN was driven by three factors. First, ResNet50 offers a well-understood bias-variance trade-off for medium-scale face datasets — deep enough to capture discriminative facial geometry, but not so deep as to overfit on the relatively small per-identity sample counts typical of real deployments. Second, PyTorch's torchvision provides a pre-trained ResNet50 checkpoint, enabling transfer learning that dramatically reduces the labelled data requirement. Third, the backbone is frozen during fine-tuning (confirmed by `TestSiameseEncoder.test_backbone_frozen`), which prevents catastrophic forgetting and keeps training fast enough for the weekly re-training cadence.

4.2 Training Datasets — LFW + VGGFace2

Fine-tuning on **LFW (Labeled Faces in the Wild)** provides exposure to the canonical benchmark distribution of celebrity face pairs, including a wide variety of lighting conditions and partial occlusions. **VGGFace2** contributes large-scale diversity across poses, ages, and ethnicities — critical for a production deployment that must handle real-world appearance variation. The combination ensures that the encoder learns embeddings that generalise well beyond any single dataset's distribution.

4.3 Siamese Architecture with Triplet Loss

A **Siamese Network** with a shared-weight encoder is the natural choice for face verification because the task is fundamentally metric learning — determining whether two face images belong to the same identity. The encoder maps a face image to an L2-normalised embedding; similarity is computed as cosine distance. Triplet loss with configurable hard/semi-hard mining pushes same-identity embeddings together and separates different-identity embeddings by a learned margin. The cosine threshold controlling the accept/reject boundary is configurable at runtime without redeployment.

4.4 Optuna Hyperparameter Optimisation

Rather than fixed hyperparameters, each re-training run can invoke `sweep_optuna.py`, which uses **Optuna's TPE sampler** to search the joint space of learning rate, margin, embedding dimension, and batch composition (P identities $\times$ K samples per identity). Each trial is a full training run logged to MLflow. The best trial by validation loss is registered as the candidate model. This means the model adapts not only its weights but also its hyperparameter configuration to the evolving distribution of registered identities.

5 Programming Paradigm — OO vs Functional

The codebase adopts a **hybrid paradigm**, blending Object-Oriented and Functional styles in a Pythonic manner appropriate to each context. This was a deliberate design choice, not an accident of style.

5.1 Object-Oriented Components

Core ML components are class-based. `LFWIdentityDataset`, `LFWPairDataset`, and `PKSampler` in `dataloader.py` extend PyTorch's Dataset and Sampler abstractions — the standard PyTorch idiom for data pipelines. The Flask application uses class instances for thread-safety (`_model_lock`, `threading.Event`).

MediaPipe's FaceDetection is instantiated and managed as an object lifecycle with explicit resource cleanup via the signal handler.

5.2 Functional / Procedural Components

Pure functions handle data transformations (`get_transform()`, `scan_lfw()`, `split_identities()`), inference (`get_embedding()`, `db_recognize()`), and configuration resolution (`resolve_params()`, `apply_params()`). The Airflow DAG tasks are all Python callables composed via `PythonOperator` — a functional composition pattern well suited to the stateless nature of DAG tasks.

5.3 Design Rationale

PyTorch idioms strongly favour class-based Dataset/Model definitions — the framework's training loop expects objects with `__len__`, `__getitem__`, and `forward()` methods. Flask routes and utility functions are naturally functional: they receive a request, perform a transformation, and return a response with no retained state. Forcing either paradigm uniformly across both domains would introduce unnecessary boilerplate. The separation leads to **high cohesion within each module** and makes the codebase immediately navigable for developers familiar with either PyTorch or Flask conventions.

6 Key Operational Behaviours

The following subsections describe each significant operational behaviour — what happens, why it was designed that way, and the specific implementation mechanism.

6.1 Face Registration & Image Persistence

During registration the user presents their face to the camera. MediaPipe detects the face bounding box, the crop is passed through ResNet50 to produce an L2-normalised embedding, and the embedding is stored in the SQLite `faces` table keyed by name. **Critically, the raw captured image is also saved to disk** under the person's directory. This design choice ensures that each registered identity contributes real-world appearance data to subsequent fine-tuning rounds — the training pipeline merges these images into the training set via `merge_misclassified_into_train()`. Without this, re-training would rely solely on benchmark datasets and could not adapt to the specific enrolled population.

6.2 Local Live Inference

Inference is performed **entirely on the local host** (or local Docker environment). No frames, crops, or embeddings are transmitted to an external server during recognition. The full pipeline — MediaPipe face detection → ResNet50 embedding extraction → cosine similarity search across the SQLite embedding store — runs within the flask-api container. This design choice prioritises **low latency and data privacy**: users' biometric data never leaves the deployment environment.

6.3 Dual-Trigger Re-training Policy

Automated re-training is triggered by one of two independent conditions. **Condition 1 — Misclassification threshold breach:** when the count of HITL-reported misclassifications exceeds a configurable threshold, the Flask API authenticates with Airflow via JWT and triggers the `face_recognition_pipeline` DAG immediately. This provides *responsiveness* to sudden quality degradation. **Condition 2 — Weekly schedule:** the DAG runs on a weekly cadence regardless of threshold, provided at least one misclassification has been logged since the last run. This provides *regularity*, keeping the model current with gradual appearance changes even in low-traffic deployments. The dual-trigger policy balances responsiveness with computational cost.

6.4 Remote GPU Support for Training

Both initial training and re-training support execution on a remote GPU server, controlled by the `RUN_REMOTE` flag in the project's `.env` file. When `RUN_REMOTE=true`, the training pipeline SSHes into the configured remote host (`REMOTE_HOST`, `REMOTE_USER`, `REMOTE_PASSWORD`) and dispatches the training job there. This allows the inference plane to remain deployed on lightweight hardware while offloading computationally intensive training to a GPU cluster.

6.5 Post Re-training: Misclassification Cleanup

After a successful re-training run and model promotion to Production, all misclassification records are **deleted** from the SQLite audit table. This prevents stale error data from influencing future re-training cycles and resets the threshold counter. Each re-training window is therefore based only on freshly observed errors, avoiding a positive-feedback loop where already-corrected errors continue to bias the training data.

6.6 Post Re-training: Embedding Re-computation

Following model promotion, the system performs a **full re-embedding pass** over all saved registration images for every enrolled person. Switching to a new model version introduces an *embedding shift* — the same face image maps to a different vector in the updated embedding space. Without re-computation, stored embeddings would be inconsistent with the active model and recognition accuracy would degrade immediately after each re-training cycle. Re-computation is performed per-person with independent error handling so that one corrupt directory does not abort the re-embedding of others.

7 Loose Coupling — Frontend vs Backend Inference Engine

A core design requirement is that the frontend UI and the backend inference engine must be **independent software blocks connected only via configurable REST API calls**. The table below demonstrates full compliance across every stated criterion.

Criterion	Evidence
Separation of concerns	index.html communicates via REST only (fetch() calls). No shared state or direct function calls between UI and inference engine.
Configurable API base	All env vars (MLFLOW_TRACKING_URI, AIRFLOW_BASE_URL, THRESHOLD) are injected via docker-compose.yaml — no code changes needed to reconfigure.
Frontend independence	index.html can be served by any static host. The Flask backend at :8080 is the sole dependency. CORS can be trivially enabled.
Backend independence	The inference engine operates independently of any UI. /metrics is consumed by Prometheus; /video_feed by any MJPEG client.
REST-only coupling	All communication between frontend actions (register, delete, flag_retrain) and backend state machine is mediated by HTTP POST/GET with JSON payloads.

Table 3 — Loose coupling compliance matrix

8 Overall Assessment

The system demonstrates a mature, production-oriented MLOps architecture with strong adherence to software engineering best practices. Design principles are comprehensively addressed: the HLD and LLD are clearly specified with block diagrams, the architecture follows microservices separation, and the REST-only coupling between the UI and inference engine satisfies all rubric criteria. The ResNet50 backbone fine-tuned on LFW and VGGFace2 provides a strong embedding foundation. The system's operational behaviours — local live inference, dual-trigger re-training policy, remote GPU support, post-retrain misclassification purge, and full embedding re-computation — collectively deliver a robust, self-improving face recognition pipeline suitable for production deployment.

The companion documents (**LLD Document** and **Test Plan & Test Cases**) provide the detailed Docker service architecture with component-level diagrams, complete REST API endpoint specifications, and unit/integration test coverage that together complete the full Design & Implementation Review submission.